Week 6

Proximal Policy Optimisation

An Elementary and Formal Introduction

Mirko Degli Esposti · Dipartimento di Fisica · Università di Bologna

§1 Prerequisites and Notation
§2 From Value Functions to Policies
§3 The Policy Gradient Theorem
§4 The Importance-Sampling Surrogate
§5 Trust-Region Motivation (TRPO → PPO)
§6 The Clipped Surrogate Objective
§7 Advantage Estimation and GAE
§8 The Complete Algorithm
§9 Hyperparameters · §10 Physics Analogies

§1

Prerequisites and Notation

MDP, policy, value functions, advantage

§1 — The MDP and Key Notation

MDP = (S , A , P , R , γ) — the 5-tuple underlying every RL problem

S	State space (may be continuous — e.g. $\mathbb{R}^4$ for CartPole)
A	Action space (discrete or continuous)
P(s'|s,a)	Transition kernel (unknown to the agent)
R(s,a)	Reward function, values $r \in \mathbb{R}$
γ ∈ [0,1)	Discount factor — controls planning horizon

Key Symbols

πθ(a|s)	Stochastic policy parameterised by θ
Vπ(s)	State-value: Eπ[Gt | st=s]
Qπ(s,a)	Action-value: Eπ[Gt | st=s, at=a]
Aπ(s,a)	Advantage: Qπ(s,a) − Vπ(s)
Gt	Discounted return: Σk≥0 γk rt+k
J(θ)	Objective: Eπθ[G0] — maximise this

§1 — The Advantage Function

The teaching signal of policy gradient

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$$

How much better is action a, compared to the average action at state s?

Zero-mean property

$E_{a\sim\pi}[\, A_\pi(s,a) \,] = 0$ for all s

Averaging advantage over the policy gives zero — it is a pure relative measure, not an absolute scale.

A > 0 → good action

Action a is better than average under π.

The policy gradient will push θ to increase π(a|s). This is the 'positive reinforcement' branch.

A < 0 → bad action

Action a is worse than average under π.

The policy gradient will push θ to decrease π(a|s). This is the 'negative reinforcement' branch.

KEY INSIGHT

The advantage replaces the raw return G_t in the policy gradient. Since $E_a[A]=0$, the baseline $V_\pi(s)$ contributes zero gradient — but dramatically reduces variance. This substitution is what makes actor-critic methods stable.

§2

From Value Functions to Policies

Why policy gradients? Why not DQN everywhere?

§2 — Two Families of RL Algorithms

Value-Based (e.g. DQN)

What is learned?

$Q^*(s,a)$ — the optimal action-value function

Policy extraction

$\pi(s) = \text{argmax}_a\, Q^*(s,a)$ (implicit, deterministic)

Action space

Discrete and enumerable (cannot scale to continuous)

Key idea

Minimise Bellman residual until $Q \to Q^*$

Limitation

No direct optimisation of $J(\theta)$; policy is a byproduct

Policy Gradient (e.g. PPO)

What is learned?

$\pi\theta(a|s)$ — the policy itself, parameterised by θ

Policy extraction

$\pi\theta$ is directly used; stochastic policy is natural

Action space

Continuous or discrete — no argmax needed

Key idea

Gradient ascent on $J(\theta) = E\pi\theta[G0]$

Limitation

High-variance gradients; careful variance reduction needed

Physics analogy — Value-based: find the ground state of H (minimise energy). Policy gradient: variational principle — optimise over trial wavefunctions $\pi\theta$ to maximise expected return.

§3

The Policy Gradient Theorem

The log-derivative trick and the REINFORCE identity

§3 — The Log-Derivative Trick

How to differentiate through trajectory probabilities

Goal: compute $\nabla\theta\, J(\theta)$ without differentiating through the unknown dynamics $P(s'|s,a)$.

Step 1 — Trajectory probability

$$P(\tau;\theta) = \rho_0(s_0) \prod_{t\geq 0} \pi_\theta(a_t|s_t)\, P(s_{t+1}|s_t, a_t)$$

$$J(\theta) = \int P(\tau;\theta)\, G(\tau)\, d\tau$$

Step 2 — Log-derivative identity (the REINFORCE trick)

$$\nabla\theta\, P(\tau;\theta) = P(\tau;\theta)\, \nabla\theta \log P(\tau;\theta)$$

Since $\log P(\tau;\theta) = \log \rho_0(s_0) + \Sigma_t \log \pi_\theta(a_t|s_t) + \Sigma_t \log P(s_{t+1}|s_t,a_t)$ and dynamics terms do NOT depend on θ:

$$\nabla\theta \log P(\tau;\theta) = \Sigma_{t\geq 0}\, \nabla\theta \log \pi_\theta(a_t|s_t) \quad \leftarrow \text{only policy terms survive!}$$

Consequence: $\nabla\theta\, J(\theta)$ depends only on $\nabla\theta \log \pi_\theta(a|s)$ — which is computable from the neural network, even though the environment dynamics $P(s'|s,a)$ are unknown and never differentiated through.

§3 — Policy Gradient Theorem & REINFORCE

Statement and fundamental limitation

POLICY GRADIENT THEOREM

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) \quad G_t]$$

$$= E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) \quad A_\pi(s_t, a_t)] \quad \leftarrow \text{variance-reduced form}$$

Second form: replacing G_t by $A_\pi(s_t,a_t)$ does not bias the gradient (baseline V_π contributes zero in expectation) but dramatically reduces variance.

REINFORCE (Williams, 1992) — the simplest implementation

1. Collect a full episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ under π_θ
2. Compute returns: $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ for each step t
3. Update: $\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) G_t$

FATAL LIMITATION — HIGH VARIANCE

The gradient estimate has extremely high variance because G_t sums all future stochastic rewards. In CartPole: G_t ranges from 1 to 500 — a 500× range per episode. This slows convergence drastically and can prevent learning entirely. → PPO addresses this on two fronts: (1) variance reduction via GAE, (2) data reuse via importance sampling.

§4

The Importance-Sampling Surrogate

Reusing off-policy data safely

§4 — Importance Sampling and the CPI Surrogate

Problem: after a gradient step $\theta \leftarrow \theta + \Delta\theta$, all collected trajectories are stale (generated by old θ) — they must be discarded. This makes vanilla policy gradient extremely sample-inefficient.

Remedy — Importance Sampling

$$E_{x\sim p}[\, f(x) \,] = E_{x\sim q}[\, (p(x)/q(x))\, f(x) \,] \quad \leftarrow \text{ evaluate } E_p \text{ using data from } q$$

Applying this to the policy gradient with $p = \pi\theta$ and $q = \pi\theta_old$:

PROBABILITY RATIO

$$r_t(\theta) = \pi\theta(a_t|s_t) \;/\; \pi\theta_old(a_t|s_t)$$

$r_t(\theta_{old}) = 1$	At the start of each update — no correction needed yet
$r_t(\theta) > 1$	$\pi\theta$ assigns higher probability to at than the old policy
$r_t(\theta) \ll 1$	$\pi\theta$ assigns much lower probability — action is being suppressed

CPI surrogate: $L^{CPI}(\theta) = E_t[\, r_t(\theta)\, \hat{A}_t \,]$ — valid only when θ is close to θ_{old}. If the update is large, r_t diverges, importance sampling breaks down, and the policy can catastrophically collapse. This is what PPO's clipping prevents.

§5

Trust-Region Motivation

From TRPO to PPO — cheaper, same idea

§5 — TRPO and the Trust-Region Idea

Schulman et al., 2015 → 2017

TRPO — Constrained Optimisation

$$\max \; L^{CPI}(\theta) \quad \text{subject to} \quad E_t[\, KL(\, \pi_{\theta_{old}} \,\|\, \pi_\theta \,)(s_t) \,] \le \delta$$

$E[KL] \le \delta$ defines a trust region — a ball around θ_{old} in KL-divergence space within which the importance-sampling approximation remains valid. Theory guarantees monotone policy improvement.

Physics analogy — Trust region ↔ Renormalisation Group step. In the RG, integrate out modes within a momentum shell $[\Lambda', \Lambda]$ — never the full theory at once. Each step is perturbatively valid. Making δ smaller ↔ smaller shell ↔ more conservative flow. The KL constraint is the exact RG analogue.

Why TRPO is impractical for deep RL:

Computational cost $O(d^2)$

The constraint requires the Hessian of the KL divergence — an $O(d^2)$ matrix for d parameters. Neural nets have millions of parameters; this is completely impractical.

Implementation complexity

The constrained problem is solved via conjugate-gradient / line-search — requires careful engineering and is incompatible with standard deep learning frameworks (PyTorch / TF).

PPO achieves similar performance to TRPO by clipping the probability ratio $r_t(\theta)$ instead of enforcing a KL constraint — same idea, first-order optimiser only, no Hessian.

§6

The Clipped Surrogate Objective

The core of PPO — formal definition and case analysis

§6 — The PPO Clipped Objective

Schulman et al., arXiv:1707.06347

PPO CLIPPED OBJECTIVE

$$L^{CLIP}(\theta) = E_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right]$$

$r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$ · $\varepsilon \approx 0.2$ (clip threshold) · $\mathrm{clip}(x,a,b) = \min(\max(x,a),b)$

Why the min + clip combination works:

Case	Unconstrained max	Clipped behaviour	Effect
$\hat{A}_t > 0$ (good action)	Increase $r_t(\theta)$ without limit	r_t capped at $1+\varepsilon$	Cannot over-reward — policy won't diverge from π_{old} in a beneficial direction
$\hat{A}_t < 0$ (bad action)	Decrease $r_t(\theta)$ without limit	r_t floored at $1-\varepsilon$	Cannot over-penalise — won't suppress an action too aggressively
$r_t = 1$ (no change)	$L = \hat{A}_t$ (baseline)	$L = \hat{A}_t$ (same)	No clipping triggered; gradient equals vanilla policy gradient exactly

The min operator is a pessimistic bound. Moves that keep r close to 1 are unhindered. Moves that push r beyond $[1-\varepsilon, 1+\varepsilon]$ hit the clip and receive zero gradient — an automatic trust region with no Hessian required.

§6 — The Full PPO Objective

Three terms: clip + value function + entropy

$$L^{PPO}(\theta) = E_t\left[L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 H[\pi_\theta](s_t) \right]$$

$L^{CLIP}(\theta)$

c_1 = —

Clipped Policy Loss

$$E_t\left[\min(r_t(\theta)\hat{A}_t , \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t) \right]$$

The core PPO objective. Penalises large policy changes while still improving on positive-advantage actions.

$L^{VF}(\theta)$

$c_1 \approx 0.5$

Value Function Loss

$$E_t\left[(V_\theta(s_t) - V^{targ}_t)^2 \right]$$

Mean squared error between the critic's prediction and the target return. Allows actor and critic to share a network backbone.

$H[\pi_\theta](s_t)$

$c_2 \approx 0$–0.01

Entropy Bonus

$$-\Sigma_a \ \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)$$

Entropy of the policy at state st. Adding this discourages premature convergence to a deterministic policy and encourages exploration.

Shared parameters (actor-critic): π_θ and V_θ share a common feature extractor (MLP backbone) with separate output heads. This is computationally efficient and allows both to improve from the same rollout data. $c_2 = 0$ is standard for CartPole.

§7

Advantage Estimation and GAE

Generalised Advantage Estimation — bias/variance tradeoff

§7 — Generalised Advantage Estimation (GAE)

Schulman et al., ICLR 2016

The Bias–Variance Tradeoff in Advantage Estimation

Estimator	Formula	Bias	Variance
Monte Carlo	$\hat{A}_t^{MC} = G_t - V_\theta(s_t)$	Zero (exact)	Very high (sum of many r.v.)
1-step TD	$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t)$	Depends on V accuracy	Low (one reward only)
k-step TD	$\hat{A}_t^{(k)} = \Sigma_{l<k} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t)$	Intermediate	Intermediate
GAE(λ)	Exponential blend of all k-step estimates	Controlled by λ	Controlled by λ

GAE Definition

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \Sigma_{l\geq 0} (\gamma\lambda)^l \delta_{t+l} \quad \text{where} \quad \delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$$

λ = 0

$\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
1-step TD error — low variance, high bias (depends entirely on accuracy of V_θ)

λ = 1

$\hat{A}_t = \Sigma_{l\geq 0} \gamma^l \delta_{t+l} = G_t - V_\theta(s_t)$
Full Monte Carlo advantage — zero bias, maximum variance

λ = 0.95

PPO default: gentle exponential average
keeps variance low while accepting small bias from the value function error

Physics analogy — GAE ↔ Exponential moving average in signal processing. Each δ_{t+l} is a noisy one-step signal; weights $(\gamma\lambda)^l$ suppress variance at the cost of lag. λ plays the role of the correlation length.

§8

The Complete PPO Algorithm

Actor-Critic with GAE — full pseudocode

§8 — PPO Algorithm (Actor-Critic + GAE)

PPO Pseudocode

```
Input: πθ (actor), Vθ (critic), γ, λ, ε, K, M, T

Repeat until convergence:
    1. Collect T-step rollout with πθold
          store {st, at, rt, st+1}
    2. Compute δt = rt + γVθ(st+1) − Vθ(st)
    3. GAE:  Ât = Σl≥0 (γλ)^l δt+l
    4. Targets: V^targ_t = Ât + Vθold(st)
    5. Normalise: Ât ← (Ât − μ) / σ
    6. Set θold ← θ

    7. For epoch k = 1, ..., K:
         For mini-batch B (size M) from rollout:
           rt(θ) = πθ(at|st) / πθold(at|st)
           L^CLIP = Et[min(rt Ât, clip(rt,1−ε,1+ε)Ât)]
           L^VF = Et[(Vθ(st) − V^targ_t)²]
           L = L^CLIP − c₁L^VF + c₂H[πθ]
           θ ← θ + α ∇θ L   (Adam)
```

Data collection

T steps with the current policy. T = n_steps (default 2048). Store full trajectories for GAE computation.

Advantage (step 3)

GAE(γ,λ) — exponentially weighted TD residuals. λ=0.95 balances bias and variance.

Data reuse (step 7)

K = n_epochs = 10 SGD passes over the same rollout. This is what makes PPO sample-efficient vs A2C (K=1).

Shared network

Actor πθ and critic Vθ share a common MLP backbone. Both improve from the same batch.

§9

Hyperparameters and Practical Guidance

What to set, what to watch, how to diagnose

§9 — Key Hyperparameters (SB3 defaults)

Hyperparameter	SB3 default	Meaning	Practical guidance
`n_steps (T)`	2048	Rollout length per update	Longer T → lower-variance gradient, more memory. 4096 smooths curves.
`batch_size (M)`	64	SGD mini-batch size	Must divide n_steps. Larger = more stable, slower per step.
`n_epochs (K)`	10	SGD passes per rollout	Increase to 15 for efficiency; >20 violates importance-sampling validity.
`gamma (γ)`	0.99	Discount factor	0.99 for long horizons (CartPole-500). Lower for short episodes.
`gae_lambda (λ)`	0.95	GAE trace decay	1.0 = Monte Carlo (max variance); 0 = 1-step TD (max bias).
`clip_range (ε)`	0.2	PPO clip threshold	0.1–0.3. Smaller = more conservative. Rarely needs tuning.
`ent_coef ($c_2$)`	0.0	Entropy bonus weight	0.01 helps sparse-reward tasks. 0 is correct for CartPole.
`vf_coef ($c_1$)`	0.5	Value function loss weight	0.5 standard. Increase if critic is consistently wrong.
`learning_rate (α)`	3e-4	Adam step size	3e-4 is robust. Can schedule linearly to 0 for fine-tuning.
`max_grad_norm`	0.5	Gradient clipping norm	Prevents exploding gradients. Keep at 0.5.

Common issue: reward plateau early → increase ent_coef, reduce clip_range · reward oscillates → reduce n_epochs to 5 · slow initial learning → reduce n_steps · value loss diverges → reduce vf_coef

§10

Physics Analogies — A Summary

Every PPO concept mapped to a physics counterpart

§10 — Physics Analogies Summary Table

RL / PPO concept	Physics analogue	Formal link
`Policy  πθ(a|s)`	Trial wavefunction in variational QM	$J(\theta)$ is the variational functional; θ^* maximises $\langle\psi\|H\|\psi\rangle$ analogue
`Policy gradient  ∇θJ`	Functional derivative $\delta J/\delta\pi$	Same mathematics; advantage A is the variational derivative
`Advantage  A(s,a)`	Force field $F = -\nabla U(s)$	$A = -\nabla U$ where $U = -V^*$ is the potential landscape
`Bellman operator  T`	RG transformation (contraction)	Both are contractions; fixed point = optimal V^* / IR fixed point
`Trust region  (TRPO)`	RG momentum shell $[\Lambda', \Lambda]$	Restrict updates to a shell for perturbative control each step
`PPO clipping`	Approximate trust region / soft cutoff	Gradient zeroed beyond the shell — no KL Hessian needed
`Discount factor γ`	Temporal correlation length	$\gamma\to 0$: nearest-neighbour only; $\gamma\to 1$: full long-range interactions
`GAE parameter λ`	Exponential MA window / correlation length	$\lambda\to 0$: 1-step (short range); $\lambda\to 1$: MC (long range)
`Entropy bonus H`	Temperature T / free energy	Max-entropy RL: $\pi^* \propto \exp(Q^*/\beta)$; entropy → exploration
`CartPole  V*(θ) profile`	Harmonic oscillator potential $V = kx^2/2$	Linearised pendulum near equilibrium is exactly a harmonic oscillator

Grim Trigger threshold $\delta \geq (T-R)/(T-P)$ ↔ Q-learning γ^* ↔ PPO GAE — all are the same recursive present-value argument (Week 5 → Week 7 bridge).

PPO — Key Takeaways

Log-derivative trick

$\nabla\theta J$ doesn't need the environment dynamics — only $\nabla\theta \log \pi\theta(a|s)$, which is computed by backprop.

Policy gradient theorem

$\nabla\theta J = E\pi[\nabla\theta \log \pi \cdot A]$ — advantage replaces raw return; zero-mean baseline reduces variance.

Importance sampling

$rt(\theta) = \pi\theta/\pi\theta old$ allows data reuse. Valid only near θold — diverging rt catastrophically collapses the policy.

PPO clipping

L^CLIP clips rt to $[1-\varepsilon, 1+\varepsilon]$ via the min operator — automatic trust region, no Hessian, first-order optimiser only.

GAE

$\hat{A}t^s = \Sigma l\ (\gamma\lambda)^l\ \delta t+l$ — exponential blend of TD residuals. λ interpolates between 1-step TD ($\lambda=0$) and MC ($\lambda=1$).

Full objective

$L^{PPO} = L^{CLIP} - c_1 L^{VF} + c_2 H$ — clip + value function regression + entropy bonus in one forward pass.

Data reuse

K epochs of SGD per rollout — this is the key efficiency gain over REINFORCE (K=1) and A2C (K=1).

Physics summary

$\pi\theta$ ↔ trial wavefunction · trust region ↔ RG shell · Bellman ↔ RG flow · CartPole $V^*(\theta)$ ↔ harmonic potential.